

Sotto il cofano: il Teensy e il codice di ichosynth

Cappello introduttivo · ICHOS 2026 · Taranto

Un racconto a grandi linee di **cosa** c'è dentro lo strumento e **come** ragiona il suo programma. Non serve saper programmare per seguirlo: è la mappa mentale per capire cosa stiamo costruendo.

1 · Il Teensy 4.1 — il cervello

L'ichosynth è costruito attorno a una piccola scheda chiamata **Teensy 4.1**. È un **microcontrollore**: un computer minuscolo, grande come un pollice, che non ha schermo né sistema operativo ma esegue *un solo programma*, all'infinito, appena riceve corrente.

Cosa lo rende speciale:

- **Potenza fuori scala** per le sue dimensioni: un processore **ARM Cortex-M7** a

600 MHz. **È nato per fare** audio in tempo reale** — proprio quello che ci serve.

- Lo crea **PJRC** (Paul Stoffregen) e si programma con l'ambiente **Arduino**:

stessa semplicità di un Arduino, ma con i muscoli.

- Ha uno **slot micro SD** integrato (per i campioni) e accetta la **PSRAM**:

16 MB di memoria extra che saldiamo sotto la scheda.

- Parla **USB-MIDI**: può sincronizzarsi con altri strumenti.

Da solo il Teensy non fa suono: lo affianchiamo a una **scheda audio (codec SGT5000)** che trasforma i numeri calcolati dal Teensy in un segnale analogico per le **cuffie**. E gli diamo un "volto": una **matrice di 256 LED (16×16)** che è allo stesso tempo schermo e tastiera.

In una frase: il Teensy *pensa* (calcola il suono, legge le manopole, accende i LED), la scheda audio *parla* (manda il suono in cuffia), la matrice *mostra e riceve* (display + strumento).

2 · Com'è fatto un programma per Teensy

Ogni programma Arduino/Teensy ha **due funzioni** fondamentali:

```
setup()    -> viene eseguita UNA volta, all'accensione  (la preparazione)
loop()     -> viene ripetuta all'INFINITO, per sempre   (il battito)
```

- `setup()` prepara tutto: accende le periferiche, carica i dati, costruisce

il motore audio.

- `loop()` è il battito cardiaco: legge gli ingressi, aggiorna lo schermo,

reagisce a quello che fai — decine di volte al secondo.

Nel nostro progetto il programma principale è il file `soundpauli_ni404.ino`; le impostazioni (quali pin, quali funzioni attive) stanno in `config.h`.

3 · Cosa succede all'accensione — `setup()`

All'avvio il Teensy esegue, in ordine, più o meno questo (versione semplificata di ciò che c'è nel codice):

1. **Accende la comunicazione** e controlla se al riavvio precedente c'è stato un

errore (*CrashReport*).

2. **Prepara i pin** degli encoder e dei pulsanti, e avvia la **matrice LED**.
3. **Mostra il logo** sulla matrice (`showIntro`).
4. **Aspetta la micro SD**: se manca, disegna l'icona "noSD" e resta in attesa

(`drawNoSD`). (Per questo una board "nuda" senza SD sembra riavviarsi: sta solo aspettando la scheda.)

5. **Carica i campioni** dalla SD nella PSRAM (`loadSamplePack`).
6. **Costruisce il "grafo audio"** — vedi sezione 5 — collegando lettori di

campioni, involuppi, filtri e mixer.

7. **Accende il codec audio** e riserva la memoria audio (`AudioMemory`).
8. (*nostro fork*) applica i **filtri** e inizializza l'**OLED**.
9. **Registra i pulsanti**: dice al programma cosa fare a click, doppio click,

pressione lunga...

A fine `setup()` lo strumento è pronto a suonare.

4 • Il battito — `loop()`

Da qui in poi il Teensy ripete questo ciclo, velocissimo:

```
loop():
  leggi il MIDI USB (clock e note in arrivo)
  interroga le 3 manopole e i pulsanti (chi è stato premuto/girato?)
  interpreta il gesto (sto disegnando una nota? cambiando pagina? ...)
  aggiorna la matrice LED (ridisegna la griglia)
```

Un dettaglio elegante: i gesti delle manopole (click, doppio click, pressione lunga) non vengono interpretati subito, ma valutati **insieme** ~80 millisecondi dopo l'ultimo movimento. Così il programma capisce, ad esempio, che un "doppio click" è diverso da due click separati.

5 • Il cuore invisibile: il motore audio

Qui sta la magia del Teensy. Il suono **non** viene generato dentro il `loop()`. Lavora in sottofondo la **Teensy Audio Library**, che gira **sugli interrupt** — cioè in modo indipendente, prioritario, senza che il programma principale debba occuparsene istante per istante.

Funziona come un **sintetizzatore modulare**: tanti "moduli" collegati da "cavi". Per ogni voce, il percorso del segnale è più o meno questo:

```
[campione in PSRAM] -> [lettore/player] -> [involuppo ADSR] -> [filtro lowpass] -> [mixer] -> [codec]
```

- Il **player** legge il campione dalla PSRAM (e può cambiarne l'intonazione).
- L'**involuppo** dà forma al suono nel tempo (attacco, rilascio...).
- Il **filtro lowpass** ne scurisce il timbro *(è qui che agisce il pulsante del

nostro fork)*.

- Il **mixer** somma le voci; il **codec** le trasforma in segnale per le cuffie.

Il `loop()` non "fa" il suono: si limita a **configurare i parametri** (quale campione, che volume, quanto filtro) e il motore audio fa il resto in tempo reale.

6 · La griglia 16×16 — schermo e sequencer



La matrice di LED è insieme **display** e **spartito**:

- le **colonne** sono i passi nel tempo (la sequenza che si ripete),
- le **righe** sono le **voci** (i diversi campioni), ognuna con un colore.

Una grande tabella in memoria (note[. . .]) ricorda *cosa* suona e *dove*. Un **timer hardware** fa avanzare il "cursore di riproduzione" al ritmo del **BPM** e, ad ogni passo, fa partire i campioni delle note accese. Il **MIDI clock OUT** (nostro fork) manda questo stesso tempo a strumenti esterni, per suonarci insieme.

7 · La memoria: perché serve la PSRAM

I campioni audio sono **grandi**: non entrano nella poca RAM interna del Teensy. Per questo li teniamo nella **PSRAM** (16 MB di memoria esterna saldata sotto la scheda, indicata nel codice come EXTMEM). È il motivo per cui la PSRAM è **obbligatoria**: senza, lo strumento non ha dove mettere i suoni e non parte.

8 · Le nostre aggiunte (il fork)

ichosynth parte dall'**NI404** di SP_ (soundpauli) e aggiunge:

- **OLED**: un piccolo schermo che mostra modalità, BPM, volume.
- **MIDI clock OUT**: sincronizza strumenti esterni (master del tempo).

- **Filtro lowpass per voce** su un pulsante (idea mutuata da TOERN).
- **Build a 3 encoder**: i comandi del 4° encoder dell'originale sono stati

rimappati sui pulsanti dei tre.

Tutte attivabili/disattivabili da `config.h` con semplici interruttori (`#define ... 1 / 0`).

9 · Mappa mentale dei file

File	A cosa serve
<code>soundpauli_ni404.ino</code>	il programma principale (<code>setup</code> + <code>loop</code> + tutto il resto)
<code>config.h</code>	le impostazioni: pin, quali funzioni attivare
<code>display.h</code>	lo schermo OLED (fork)
<code>audioinit.h</code>	la definizione del grafo audio (i "moduli" e i "cavi")
<code>colors.h</code> · <code>files.h</code>	colori delle voci e tabelle di supporto

In sintesi: un cervello potente (Teensy) che, in un ciclo infinito, legge le tue manopole e ridisegna una griglia di luci, mentre un motore audio invisibile trasforma campioni in musica. Tutto il resto è dettaglio — e ora sapete dov'è.

ichosynth · fork di NI404 (SP_/soundpauli) · firmware open-source MIT